

Crew Pairing Problem using Column Generation

*-Rajas Biliye,
Vivek Bhatia*

Contents

1. Introduction
2. Problem Definition
3. Terminology
4. General Formulation
5. Impediments in this Formulation
6. Restricted Master Problem (RMP)
7. Mixed Integer Linear Program (MILP)
8. Column Generation
9. Julia Implementation
10. Results
11. References

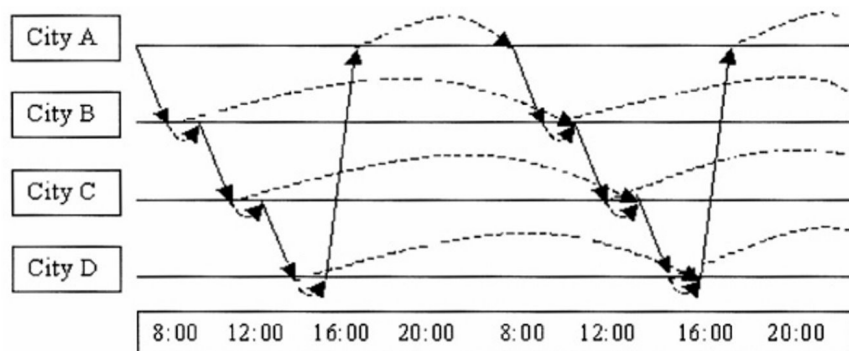
Introduction

Crew scheduling is the problem of assigning workers (a crew) to a set of tasks. Crew scheduling is an important step in any airline's operational planning process. It is directly preceded by aircraft fleet assignment. Crew scheduling problem is generally divided into two parts, crew pairing and crew assignment. Crew pairing is the problem of creating pairings for undecided crews and crew assignment is the problem of assigning those pairings to individual crews and effectively creating their itinerary for the next few days. In this project we focus on the crew pairing problem.

Problem Definition

Given a set F of scheduled flight legs, select a subset of pairings $S \subseteq P$ such that every flight f in F appears in exactly one chosen pairing, while minimizing the total cost the total repositioning and layover cost. Feasible transitions must satisfy minimum connection times (one-hour sit), maximum length of pairing (24 hours) and maximum duty-period limits (16 hours of total flying + deadhead).

Terminology



Flight

Represented by solid arrows between cities.

Duty Periods

Indicated by sequences of connected flights within a single day. These are grouped by continuous activity with legal rest in between.

Pairing

A complete multi-day trip for a crew member, including multiple duty periods.

Deadhead

Transporting crew members as passengers to reposition them for their next assignment.

General Formulation

Crew pairing models are typically formulated as set partitioning problems, in which we want to find a minimum cost subset of the feasible pairings such that every flight segment is included in exactly one chosen pairing.

Let F be the set of flight segments to be covered and let P be the set of all feasible pairings. Decision variable y_p is equal to 1 if pairing p is included in the solution, and 0 otherwise. Column p has a 1 in row i of the constraint matrix if flight i is included in pairing p and a 0 otherwise.

Sets

$$F = \{1, 2, \dots, m\} \quad (\text{flights}), \quad P = \{1, 2, \dots, n\} \quad (\text{pairings}).$$

Parameters

$$a_{ij} = \begin{cases} 1, & \text{if flight } i \text{ is in pairing } j, \\ 0, & \text{otherwise,} \end{cases} \quad i \in F, j \in P,$$

$$c_j = (\text{layover cost in pairing } j) + (\text{deadhead cost in pairing } j).$$

Decision Variables

$$y_j \in \{0, 1\}, \quad j \in P,$$

where

$$y_j = \begin{cases} 1, & \text{if pairing } j \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

Model

$$\begin{aligned} \min \quad & \sum_{j \in P} c_j y_j \quad (\text{minimize total pairing cost}) \\ \text{s.t.} \quad & \sum_{j \in P} a_{ij} y_j = 1, \quad \forall i \in F \quad (\text{cover each flight exactly once}), \\ & y_j \in \{0, 1\}, \quad \forall j \in P. \end{aligned}$$

Impediments in this Formulation

Combinatorial Explosion

The above formulation requires enumeration of all the pairings. A domestic problem on a hub-and-spoke network with several hundred flights typically has billions of pairings.

Complex Legal and Operational Constraints

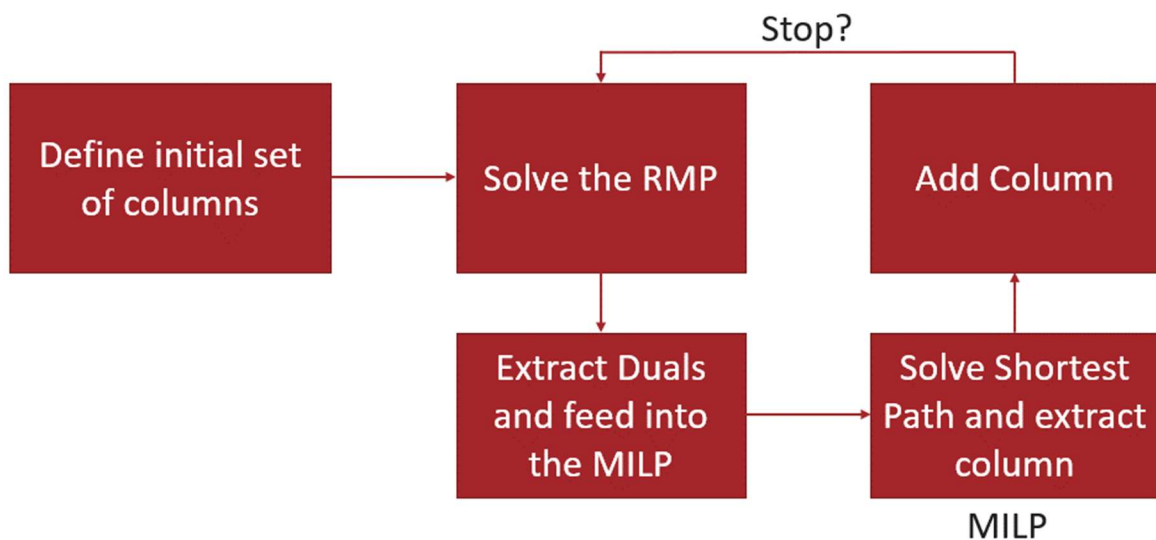
Pairings are further constrained by a complex array of rest requirements, flying time restrictions, and other constraints. Example- the 8-in-24 rule.

Nonlinear and Nonuniform Cost Structures

In the real world setting the cost structure yields a nonlinear, piecewise cost function that is different for each pairing.

Column Generation to the rescue!

Enumerating all feasible pairings is computationally intractable for large-scale problems due to the exponential growth in the total number of pairings (Combinatorial Explosion). Column generation addresses this by iteratively adding only promising pairings (columns) that improve the current solution.



Restricted Master Problem (RMP)

The RMP is the linear-programming relaxation of the full crew-pairing set-partitioning model, solved over a manageable subset of pairings to enforce that each flight is covered exactly once.

Mixed Integer Linear Program (MILP)

The MILP chooses exactly one $s \rightarrow t$ route through the flight network that minimizes the negative reduced cost to add that pairing into the RMP.

Column Generation

The column with the least reduced cost is added to the RMP.

STOP Condition

The algorithm stops when no columns with negative reduced cost can be found.

Restricted Master Problem (RMP)

The Restricted Master Problem (RMP) is the linear-programming relaxation of the full crew-pairing set-partitioning model in which, instead of enumerating every conceivable pairing, we work with a manageable subset of columns. In the RMP, each pairing in the current column pool has a nonnegative decision variable y and known cost c_p , and we enforce one “cover-exactly-once” constraint for each flight.

A typical warm-start seeds the RMP with “single-flight” columns $s \rightarrow f_i \rightarrow t$ for each flight i , ensuring initial feasibility and a valid simplex basis.

This initialization produces reasonable duals π_f up front, stabilizing early reduced-cost calculations before longer multi-leg pairings are generated.

Mixed Integer Linear Program

Once the RMP is solved and the duals (shadow prices) are obtained for each flight coverage constraint, the Mixed Integer Linear Program is used to find the pairing with the most negative reduced cost on the flight network.

The network is a directed acyclic graph with nodes $\{s\} \cup F \cup \{t\}$ and whose arcs (i, j) represent either

1. a **layover** connection if flight i arrives at the same airport where flight j departs with at least 1 hour in between, or
2. a **deadhead** connection if there is enough time to reposition from flight i 's arrival airport to flight j 's departure airport plus 1 hour to accommodate other circumstances, or
3. a **start** arc (s, f) or **end** arc (f, t) .

The source(s) and sink(t) are the crew-base. This structure enables us to only create pairings that start and end at the crew base. Each arc (i, j) has an original “cost” c_{ij} (ground time $\times$ layover-rate + reposition hours $\times$ deadhead-rate). Its reduced cost is $\tilde{c}_{ij} = c_{ij} - \pi_j$.

If the optimal objective is **negative**, the corresponding $s \rightarrow t$ path defines a new pairing of flights whose true cost minus the sum of the π_j is negative. We then add that column to the RMP. When no path of negative reduced cost remains, the LP relaxation of our full set-partitioning master problem is solved to optimality, and we stop generating new pairings.

Pricing Subproblem: Shortest-Path with Reduced Costs

Sets and Indices

$$F = \{1, 2, \dots, m\}, \quad s \text{ (source)}, \quad t \text{ (sink)}.$$

$$A = A_{\text{lay}} \cup A_{\text{dh}} \cup A_{s \rightarrow F} \cup A_{F \rightarrow t},$$

where

$$A_{\text{lay}} = \{(f, g) \in F \times F : \text{same-airport \& 1h min turn}\}, \quad A_{\text{dh}} = \{(f, g) \in F \times F : \text{deadhead feasible}\},$$

$$A_{s \rightarrow F} = \{(s, f) : f \in F\}, \quad A_{F \rightarrow t} = \{(f, t) : f \in F\}.$$

Parameters

$$c_{fg} \in R \quad \forall (f, g) \in A, \quad \pi_f \in R \quad \forall f \in F.$$

Decision Variables

$$x_{fg} \in \{0, 1\} \quad \forall (f, g) \in A.$$

Objective

$$\min_x \sum_{(f,g) \in A} (c_{fg} - \pi_g) x_{fg}.$$

Constraints

$$\sum_{f \in F} x_{sf} = 1, \quad (\text{start exactly once at source})$$

$$\sum_{f \in F} x_{ft} = 1, \quad (\text{end exactly once at sink})$$

$$\sum_{\substack{i \in \{s\} \cup F \\ (i,n) \in A}} x_{in} = \sum_{\substack{j \in F \cup \{t\} \\ (n,j) \in A}} x_{nj}, \quad \forall n \in F \quad (\text{flow conservation})$$

$$x_{sf} + x_{gt} \leq 1, \quad \forall (f, g) \in I \quad (\text{incompatibility})$$

$$\sum_{(i,j) \in A} (\ell_{ij} + d_{ij}) x_{ij} \leq 16, \quad (\text{max duty time})$$

$$x_{fg} \in \{0, 1\}, \quad \forall (f, g) \in A.$$

Column generation

Column generation is an optimization technique used to solve large-scale linear programs with too many variables (columns) to include explicitly. Instead of solving the full problem, it starts with a small subset of variables and dynamically generates more only when needed.

We begin with a handful of obvious pairings—one per flight—solve that reduced LP to optimality, and extract the dual prices for each flight-coverage constraint. Those duals then feed into a separate pricing subproblem (in our case, a reduced cost shortest path model) that searches for any new pairing whose cost minus its associated duals is negative; if one is found, it must improve the RMP objective, so we add it as a new column. We repeat solve-price-add until the pricing subproblem proves there are no more negative reduced cost columns, at which point the current RMP solution is guaranteed optimal for the full LP.

Julia Implementation

Refer to the Jupyter Notebook.

1. Data-and-Network Generation

Sets up a synthetic hub-and-spoke flight network for one week. Number of airports and number of flights need to be specified, each flight is randomly assigned to go either hub→spoke or spoke→hub, and each flight draws a random flight duration (1–5 hours) and departure time so that its arrival is within the horizon. The deadhead time is calculated as flight time between two airports plus 1 hour. We have considered a model with 1 hub airport.

The instances considered are 1. 6 airports, 50 flights; 2. 8 airports, 100 flights; 3. 12 airports, 150 flights

2. Restricted Master Problem (RMP)

The next function, `build_rmp`, constructs the Restricted Master Problem: Here, each column represents a candidate pairing (an ordered sequence of flights), and each pairing has an associated cost. We introduce one variable y_j per pairing (continuous if relax-and-price, or binary for the final integer solve), and impose exactly-once coverage constraints that force the sum of the y_j variables covering each flight to equal one. The objective is simply to minimize the total pairing cost. This builder takes as input the list of currently generated pairings (initially each single-flight loop) and their costs, solves the model, and returns both the primal solution and the dual prices (shadow values) for the coverage constraints. Those duals will serve as the incentive signals in the subsequent pricing step.

3. Pricing Subproblem: Reduced-Cost Shortest Path

With the dual prices π_f from the RMP, the `solve_shortest_path` function solves the pricing subproblem. It creates a flight-network shortest-path MIP in which nodes correspond to flights (plus a single source “crew base” and sink “crew base”), and arcs represent either feasible layovers, deadhead moves, or start/end connections at the hub. The model enforces flow conservation, exactly one arc out of the source and one into the sink, a maximum total “duty time” of 16 hours, and additional incompatibility cuts that limit the pairing length to 24 hours. Crucially, each arc’s objective coefficient is its true cost minus the dual price of its head flight, so solving for the minimum reduced-cost path identifies any pairing with negative reduced cost. The optimal pairing is then returned to be added as columns in the next RMP iteration.

4. Column Generation Loop

Starting from a trivial warm-start of single-flight pairings, we alternate between (1) solving the RMP to optimality—thereby obtaining new duals—and (2) calling the pricing function to create a new pairing whose reduced cost is negative. Whenever the pricing step yields one such pairing, we translate each returned path back into a flight-sequence pairing, compute its actual cost, and append it to our column pool. The loop repeats until the pricing subproblem can no longer find any pairing with negative reduced cost, at which point the LP relaxation of the full set-partitioning model is optimal over all considered pairings.

5. Final Integer Solve

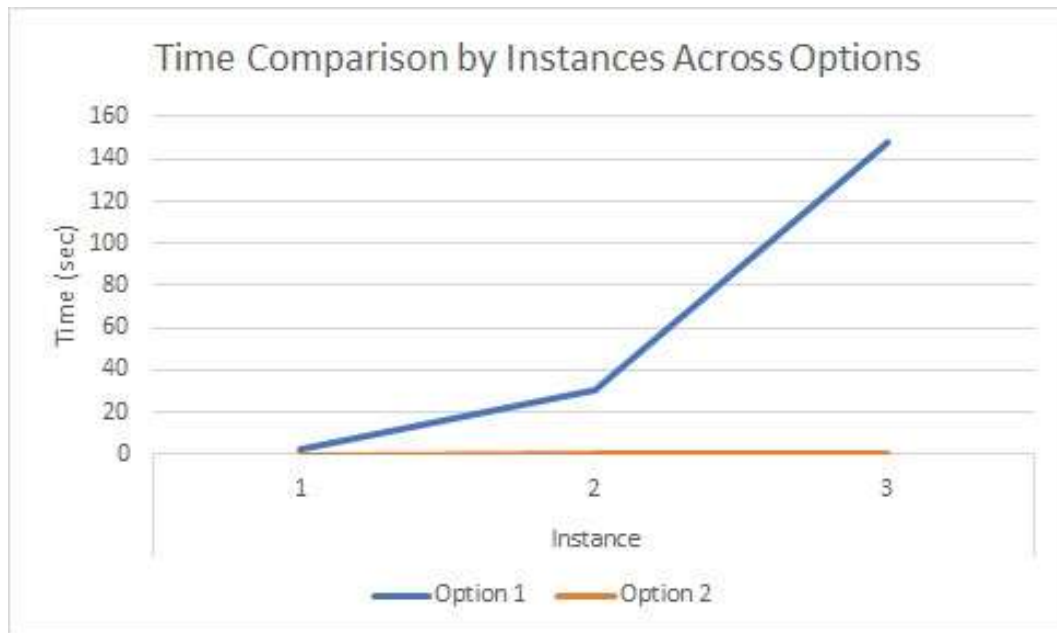
Once no further improving columns remain, we perform one last call to `build_rmp`, this time enforcing binary integrality on the `yj` variables. This integer program selects a minimum-cost subset of the generated pairings such that each flight is covered exactly once. Solving this final model yields the actual crew-pairing solution.

6. Improvement

As an experimental enhancement, we've turned on Gurobi's solution-pool feature in the pricing subproblem—by setting `PoolSearchMode = 2` and `PoolSolutions = 20` so that when we solve the reduced-cost shortest-path MILP we actually harvest all of the top 20 (or fewer) distinct paths whose objective (reduced cost) is negative and add them to the RMP.

Traditional column generation solves the pricing problem once per iteration, adds the single best negative-reduced-cost column to the RMP, and repeats—a process that can require dozens or even hundreds of master-problem solves when many near-optimal columns lurk just below zero. By turning on Gurobi's solution-pool, we instead ask the solver to return up to twenty distinct feasible paths in one go. After optimization we iterate through the pool—using `PoolObjVal` to identify each solution's reduced cost and `Xn` to reconstruct its arcs—and harvest all columns whose reduced cost is meaningfully negative. Injecting this batch of improving columns at once typically slashes the number of RMP re-solves needed, because we capture both the very best and the feasible alternatives simultaneously. Although exploring a solution pool adds overhead to each pricing solve, that cost is typically outweighed by the dramatic reduction in master-problem iterations, yielding large net speed-ups on richly connected pairing networks.

Results



Below are the summarized results

Time			
		Option 1	Option 2
Instance	1	2.754	0.006
	2	30.72	0.011
	3	148.352	0.028

Iterations			
		Option 1	Option 2
Instance	1	37	3
	2	78	6
	3	161	6

1. Multi-column pricing drastically reduces solve time

By harvesting all negative-reduced-cost pairings in one Gurobi call—rather than solving the subproblem repeatedly for a single column—each MILP solve spreads the solver’s fixed startup time across multiple new columns, significantly reducing the overall runtime.

2. Fewer iterations to convergence

Injecting every improving pairing at once shrinks the column-generation loop: because each iteration adds all viable columns, the RMP reaches optimality in far fewer rounds, cutting down the number of master-problem solves needed.

References

- Barnhart et al. (2003) *Airline Crew Scheduling*
- Klabjan et al. (1999) *Solving Large Airline Crew Scheduling Problems: Random Pairing Generation and Strong Branching*